

State estimation in the simulator

Lab experiment 2 — Kalman filter for GPS, odometry and IMU

Supervision: robotics lab

Version 0.1 — winter semester

You estimate, you do not drive. A grader places your robot at the start, drives it through a fixed command sequence and records both the exact pose (**truth**) and your estimate **kf/pose**. Grading is on accuracy: RMSE against the truth, improvement ratio over the raw sensor, maximum error during a GPS outage and consistency of the standard deviation you publish (NEES). For that you build a Kalman filter with a state vector of position and velocity, feed it GPS (slow, noisy, sometimes missing), odometry (fast, drifting) and IMU (clean short-term, worthless long-term), and report *how* uncertain you are. The simulator is the one from experiment 1 — new are an IMU noise model, three topics and the question of whether your numbers are right.

Contents

1. Experiment goal and prelab questions	2
1.1. Goal	2
1.2. Prelab questions (compute these before the lab session)	2
2. Theory: predict, measure, weigh	3
2.1. State, prediction, update, covariance	3
2.2. The CV model in matrices	4
2.3. Why the gyro helps short-term and odometry long-term	4
2.4. GPS: noise, bias, GPS outage, model error	4
3. The simulator	6
3.1. New topics and what they carry	6
3.2. The IMU model and its numbers	6
3.3. Evaluation in the GUI	7
4. Getting started	8
4.1. Installation	8
4.2. Workshop: running your own filter	8
4.3. Running under ROS 2 with the launch file	8

4.4. Three rules that save the run	9
5. Tasks K1 to K4	10
6. Evaluation and lab report	11
A. Formula sheet	12
B. Solutions to the prelab questions	12
C. Topic and API overview	13

1. Experiment goal and prelab questions

1.1. Goal

After this experiment you can

- predict a state vector (x, y, v_x, v_y) with a CV model and fold in every measurement with its own uncertainty — prediction, update and Kalman gain from P , Q and R , not from guesswork,
- state your own uncertainty so that it is *right*, and check it with the normalized error squared (NEES),
- use the odometry *twist* and the gyro rate as a motion model to bridge a ten-second GPS outage below 1.8 m error,
- separate drift (correlated, growing) from measurement noise (white, zero-mean) and justify why an IMU is never integrated open-loop,
- build RMSE time series, a Q/R table and a NEES discussion for the lab report from a measurement log.

Division of roles.

The grader parks the robot back at the spawn pose before *every* KF task and drives the command sequence from `config/tasks.json` itself over `cmd_vel`. Your node sends *no* wheel speeds and defines *no* `inverse_kinematics` — otherwise it competes with the grader and the run is worthless. Everyone works alone: one robot name, one filter node.

1.2. Prelab questions (compute these before the lab session)

Use the simulator values from table 2 (`mecanum_lab/types.py: DEFAULT_CONFIG`) and the test profiles from `config/tasks.json`: noise density $\sigma_a = 0.002 \text{ m/s}^2/\sqrt{\text{Hz}}$, accelerometer bias $b_a = 0.05 \text{ m/s}^2$, IMU rate 100 Hz, wheel noise $\sigma_{\text{wheel}} = 0.04 \text{ rad/s}$, GPS spread 0.5 m. Solutions: appendix B.

1. **What dominates the IMU?** A noise density of $0.002 \text{ m/s}^2/\sqrt{\text{Hz}}$ at 100 Hz: what standard deviation does *one* sample have, and why does it read $\sqrt{\text{rate}/2}$? Compute the position error that a fixed bias of 0.05 m/s^2 produces after 10 s of double integration,

and compare it with the contribution of the white noise after the same time. What follows for filter practice?

2. **Recursion by hand.** One-dimensional CV model with state $(x, v)^\top$, $F = \begin{pmatrix} 1 & dt \\ 0 & 1 \end{pmatrix}$, $H = \begin{pmatrix} 1 & 0 \end{pmatrix}$. Given: $\hat{x} = 2.0$ m, $\hat{v} = 0$, $P = \text{diag}(0.04; 1.0)$ in m^2 and m^2/s^2 , $dt = 0.5$ s, $q = 0.6 \text{ m}^2/\text{s}^3$, $R = 0.25 \text{ m}^2$, measurement $z = 2.40$ m. Determine P^- , the gain K (both entries), $\hat{x}^+$, $\hat{v}^+$ and the position standard deviation before and after the update.
3. **Drift is not white noise.** Odometry comes from the *measured* wheel speeds, and each of them carries σ_{wheel} . Why must the odometry *pose* error not enter the filter as white measurement noise with a constant R ? What do covariance and NEES do when it is done anyway?
4. **NEES.** For one run the evaluation reports $\overline{\text{NEES}} = 0.1, 1$ and 8 . What does each of them say about the stated standard deviation (factor, which direction), and which of the three values falls inside the band for K3 ($0.05 \dots 3.5$)?

2. Theory: predict, measure, weigh

2.1. State, prediction, update, covariance

The filter knows a state vector ξ and its uncertainty, the covariance P . Here $\xi = (x, y, v_x, v_y)^\top$ in world meters is enough at first; K4 asks for one more degree of freedom for the turn rate. The cycle always runs the same way: **prediction** over the real elapsed time dt since the last state, **update** as soon as a measurement z with its uncertainty R arrives.

$$\xi^- = F \xi + \text{inputs}, \quad P^- = F P F^\top + Q \quad (1)$$

$$\mathbf{y} = z - H \xi^-, \quad S = H P^- H^\top + R, \quad K = P^- H^\top S^{-1} \quad (2)$$

$$\xi^+ = \xi^- + K \mathbf{y}, \quad P^+ = (I - KH) P^- (I - KH)^\top + K R K^\top \quad (3)$$

For a directly measured quantity $K = P^- / (P^- + R)$: if the prediction is more uncertain than the measurement, the filter pulls toward the measurement; if P is small, little happens. The two knobs mean different things: Q says what the *model* lacks (unexplained acceleration), R what the *sensor* lacks. Raising only Q makes the filter twitchy; raising only R makes it sluggish. Both set the covariance — and with it the numbers the grade is computed from (impr. = improvement ratio):

$$\text{NEES} = \frac{1}{N} \sum_i \frac{1}{2} \left(\frac{e_{x,i}^2}{s_{x,i}^2} + \frac{e_{y,i}^2}{s_{y,i}^2} \right), \quad \text{RMSE} = \sqrt{\langle e_x^2 + e_y^2 \rangle}, \quad \text{impr.} = \frac{\text{RMSE}_{\text{GPS}}}{\text{RMSE}_{\text{KF}}}. \quad (4)$$

With two degrees of freedom the expected NEES value is 1; a single value up to about 5.99 is still unremarkable under a χ^2 distribution. Without eq. (3) in Joseph form P tends to stay asymmetric after rounding errors — for a 4×4 filter by hand probably the most common silent source of NEES far above 1.

2.2. The CV model in matrices

Constant velocity is what the model is called, because it does not know acceleration. In matrices (world frame, wheel order irrelevant, state (x, y, v_x, v_y)):

$$F(dt) = \begin{pmatrix} 1 & 0 & dt & 0 \\ 0 & 1 & 0 & dt \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}, \quad H = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix}, \quad R = \begin{pmatrix} \sigma_{\text{gps}}^2 & 0 \\ 0 & \sigma_{\text{gps}}^2 \end{pmatrix}, \quad (5)$$

$$Q(dt) = q \begin{pmatrix} \frac{dt^4}{4} & 0 & \frac{dt^3}{2} & 0 \\ 0 & \frac{dt^4}{4} & 0 & \frac{dt^3}{2} \\ \frac{dt^3}{2} & 0 & dt^2 & 0 \\ 0 & \frac{dt^3}{2} & 0 & dt^2 \end{pmatrix}, \quad q \text{ in } \text{m}^2/\text{s}^3. \quad (6)$$

q is the assumed uncertainty of the *acceleration*, that is, of what the CV model cannot represent; a usable start is $0.3 \dots 1.0$, recommended in `kf.q_acc = 0.6` (table 2). σ_{gps} is not guessed but read from the sensor profile (`rob.config("gps.sigma_xy", 0.5)` or `/sim/config`). Without numpy, hand matrix multiplication is enough: eq. (5) is sparse, eq. (6) block-wise — four lines of code per axis. Two things decide good or bad: dt is the *real* difference of the message stamps (not an averaged fixed value), and the filter predicts up to the *measurement time* of the incoming measurement before it updates. At 5 Hz GPS and 0.85 m/s, 100 ms of lag costs 8.5 cm — already a third of the K4 target.

2.3. Why the gyro helps short-term and odometry long-term

Two facts determine the architecture of the filter. **First:** an IMU measures specific force, and its error grows with the integration order. The values from table 2 say: gyro noise $10^{-4} \text{ rad/s}/\sqrt{\text{Hz}}$ is $7.1 \cdot 10^{-4} \text{ rad/s}$ per sample — negligible; the gyro bias $5 \cdot 10^{-3} \text{ rad/s}$ gives 0.05 rad of heading error in 10 s (2.9°), which costs 0.25 m of cross-track error over 10 m of driving. For the accelerometer a 0.05 m/s^2 bias becomes $\frac{1}{2}bt^2 = 2.5 \text{ m}$ of position error in 10 s through *two* integrations. Hence: turn rate yes (one integration, small), acceleration only short-term and never open-loop — the IMU is a motion model for fractions of a second, not a positioning device. **Second:** odometry delivers two different things. Its *pose* is dead reckoning and drifts (random walk from σ_{wheel} , rotated by the heading error). Its *twist* (v_x, v_y, ω) , however, is a direct measurement: with $r = 0.05 \text{ m}$ and $\sigma_{\text{wheel}} = 0.04$ to 0.05 rad/s the value $v_x = \frac{r}{4} \sum_i w_i$ carries only $\approx 1 \text{ mm/s}$ of noise *without* drift. The workable filter therefore predicts with the *measured* body velocity (odometry twist, rotated into the world frame) and the gyro rate, and updates the position with GPS. Anyone who instead throws the odometry *pose* in as a measurement with a fixed R doubles information that is already in the prediction — the third way is to carry the drift as its own state value (bias) and let R grow with time.

2.4. GPS: noise, bias, GPS outage, model error

GPS is the only absolute sensor. Its noise is correlated in an unknown way, so it is practically made for R : $\sigma_{xy} = 0.5 \text{ m}$ in K1/K3, 0.8 m in K2, 0.6 m in K4 — the raw RMSE sits at $\sigma\sqrt{2} \approx 0.7 \text{ m}$ and 1.1 m , and the improvement ratio in eq. (4) is

computed against exactly that number. A *fixed* bias (`gps.bias_xy`, multi-path) is unobservable from GPS alone: no filter can remove it, but you can carry it as a constant uncertainty inside R ; `gps.bias_step` adds it on top for one time window — the outlier drill. The **GPS outage** finally (`gps.gap = [15, 8]` in K2) drains the measurement side completely: prediction keeps running, the covariance grows only through Q , and the error grows with the elapsed time — bounded by the model error, since a CV model cannot predict a turn at 0.26 rad/s. So K2 does not grade the mean RMSE first, but the maximum error inside the gap (≤ 1.8 m): anyone who takes the gyro turn rate into the state vector survives the ten seconds; anyone who runs pure CV drives straight into the arena corner.

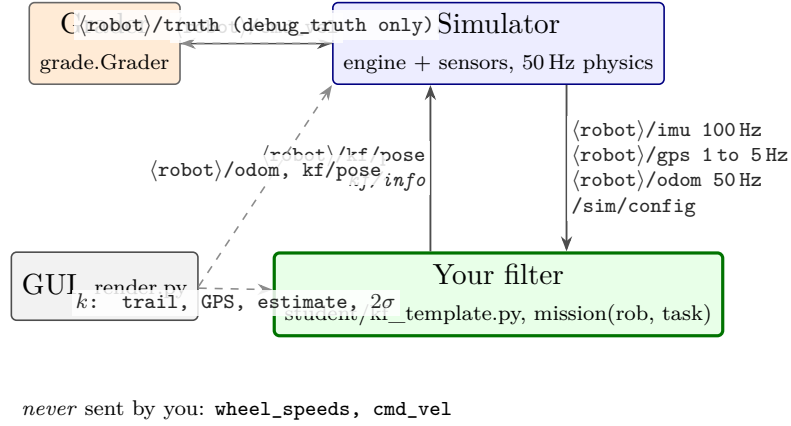


Figure 1: Signal flow of experiment 2. The grader drives, the simulator supplies the sensors, you supply `kf/pose` including standard deviation — at the measured position, not at the truth.

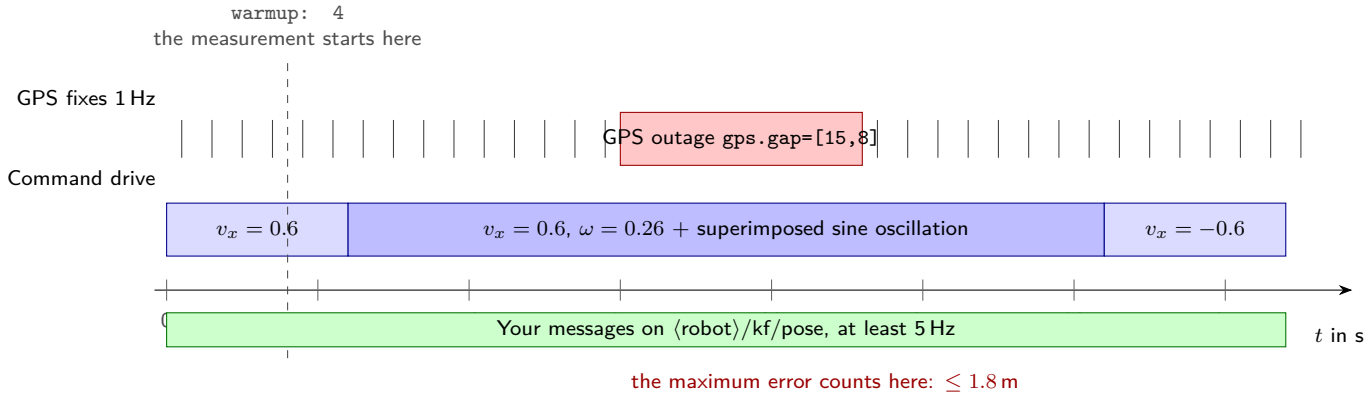


Figure 2: Timeline of task `kf_fusion`: command segments from `config/tasks.json`, GPS fixes with a gap from 15s to 23s, and the grader's measurement window starting at warmup.

3. The simulator

3.1. New topics and what they carry

Experiment 1 stays as it is; the filter builds on it. table 1 shows the access — the paths carry the robot name, `topic("kf","alice")` gives `/alice/kf/pose`.

Table 1: New and changed topics in experiment 2 (`types.MSG_SPECS`).

Topic	Type (ROS 2)	Content and direction
<code><robot>/imu</code>	<code>sensor_msgs/msg/Imu</code>	100 Hz (adjustable), axes in the body frame, <code>linear_acceleration.z</code> $\approx +g$; roll/pitch included
<code><robot>/kf/pose</code>	<code>geometry_msgs/msg/PoseWith_CovarianceStamped</code>	Variance including 1σ (diagonal) — the graded quantity
<code><robot>/kf/info</code>	<code>std_msgs/msg/String</code> (JSON)	diagnostics (runtime, rates, Q/R); the evaluator ignores it
<code><robot>/truth</code>	<code>geometry_msgs/msg/PoseStamped</code>	exact pose at the adjustable <code>truth.rate</code> , only with <code>debug_truth</code> ($= -\text{truth}$)
<code>/sim/config</code>	<code>std_msgs/msg/String</code> (JSON)	active sensor profile (<code>gps</code> , <code>odom</code> , <code>imu</code> , <code>truth</code> , <code>rate</code> , <code>seed</code>) — this is where the σ you put into R lives
<code><robot>/odom</code> , <code><robot>/gps</code>	as in experiment 1	odometry 50 Hz with <code>twist</code> ; GPS at the rate and spread of the test profile

3.2. The IMU model and its numbers

`mecanum_lab/types.py` (`DEFAULT_CONFIG`) is the source of every number; `config/default.json` and `-set` write over it, and the task’s test profile does so anyway (section 4). The “per sample” column is the quantity the filter cares about: densities are converted to the sampling rate with $\sqrt{\text{rate}/2}$, bias random walk per step with $\sqrt{1/\text{rate}}$.

Why `az` reads $\approx +9.81$ at rest. A MEMS accelerometer measures no motion but specific force — the force per mass that keeps it from falling freely. On flat ground that is the normal force pointing up, i.e. $+g$. That is neither a measurement error nor a sign bug, but the physics of the chip: anyone who integrates `az` as it is ends up 490 m high after 10 s. Only `ax` and `ay` matter in the plane — and those are biased, tilted and superimposed, which is why they are only usable short-term. `gx/gy` pick up the same tilt, `gz` is the main signal in 2D.

Measure instead of believing. A standstill run over 5 s (seed 1, robot *alice*, 502 samples) gives `az` $= 9.74 \pm 0.014 \text{ m/s}^2$ — the spread is exactly the computed density, the difference to 9.81 is the `accel_bias` of the z -axis — and a gyro mean of $-5.4 \cdot 10^{-3} \text{ rad/s}$. That mean is your calibration: average the IMU at standstill (before the drive or in every pause), subtract the mean — that is how the calibration is done later on the real robot.

Table 2: Sensor settings from DEFAULT_CONFIG (extract for experiment 2).

Quantity	Value	Meaning for your filter
<i>IMU (imu) — MEMS class, deliberately this broken</i>		
rate	100 Hz	K4 runs at 200 Hz; <code>-set imu.rate=400</code> is allowed
accel_noise	$2 \text{ e-3 m/s}^2/\sqrt{\text{Hz}}$	$\Rightarrow 14 \text{ mm/s}^2$ per sample
accel_bias	0.05 m/s^2	redrawn per robot and axis (seed); double integration: 2.5 m in 10 s
accel_bias_walk	$5 \text{ e-4 m/s}^2/\sqrt{\text{s}}$	the bias does not stay put: after 10 s it spreads by 1.6 mm/s^2
accel_scale	1 e-3 (0.1 %)	averaging does not make it disappear
gyro_noise	$1 \text{ e-4 rad/s}/\sqrt{\text{Hz}}$	$\Rightarrow 0.7 \text{ mrad/s}$ per sample
gyro_bias	5 e-3 rad/s	0.05 rad of heading error in 10 s — averaging helps
gyro_bias_walk	$2 \text{ e-5 rad/s}/\sqrt{\text{s}}$	after 10 s the bias itself spreads by 0.06 mrad/s — small, but it does not go away on its own
tilt_sigma, tilt_tau	6 mrad, 0.4 s	the suspension wobbles, gravity leaks into ax/ay: $g\varphi \approx 0.06 \text{ m/s}^2$
vibration, vibration_hz	0.08 m/s^2 , 16 Hz	chassis vibration, damp it away (low-pass filter or averaging)
startup, startup_bias	0.4 s, 0.5 m/s^2	driver settling — do not calibrate during this time
gravity	9.81 m/s^2	sits fully in az
<i>Other sensing</i>		
gps.rate, gps.sigma_xy	5 Hz, 0.06 m	experiment 1; the KF tasks use 1 to 5 Hz and 0.5 to 0.8 m
gps.bias_xy, gap, bias_step	[0, 0], null, null	offset, GPS outage $[t_0, \text{duration}]$, step bias
odom.rate, sigma_wheel	50 Hz, 0.04 rad/s	on every wheel measurement — that is what grows into the drift
truth.rate	20 Hz	rate of the exact pose (grading)
kf.rate, kf.q_acc, kf.q_turn	20 Hz, $0.6 \text{ m}^2/\text{s}^3$, 0.02	<i>recommendation to you</i> — the simulator never uses this block

3.3. Evaluation in the GUI

The display is the same as in experiment 1; the overlay of the estimate is new. The help line in the HUD: `q quit | SPACE pause | l scan | t trail | k estimate | 0 all | 1..9 robot | +/- zoom`. Key *k* toggles the GPS crosses, the estimate and the 2σ ellipse around it — that ellipse is your NEES with eyes: it must *run around* the truth, not just try to enclose it. The HUD also shows the current distance from your estimate to the truth (`kf_err`) — visible only, never used inside the filter. For the lab report the measurement series belongs in a file: `-log messung.csv` writes a semicolon line every 0.05 s of simulation time: `t, x_truth, x_gps, t_age_gps, x_kf, sx_kf, ax_imu` and more (column header from `mecanum_lab/logbook.py`); `python3 tools/kfplot.py messung.csv` draws time series, RMSE and — without matplotlib — an ASCII plot.

4. Getting started

4.1. Installation

```
./install.sh          # once: checks pygame, ROS 2 and the tests
# alternatively:  sudo apt install python3-pygame      (enough for stub mode)
./lab docs            # topics, tasks and examples at a glance
```

Tested on Ubuntu 24.04 with ROS 2 Kilted/Jazzy; without ROS everything runs over the in-process bus (MECANUM_ROS=stub) — then `ros2 topic list` of course shows nothing.

4.2. Workshop: running your own filter

```
# 1) workshop with a window: the sim drives, you estimate, truth is on
./lab run --world arena --task kf_gps --robot alice --robots alice \\\
--controller student/kf_template.py --truth --log messung.csv

# 2) grade like the submission checker (one task, or all four)
./lab grade --task kf_gps --controller student/kf_template.py --log messung.
    csv
./lab grade --task kf_all --controller student/kf_template.py \\\
--log messung.csv --json bericht.json
python3 tools/kfplot.py messung.csv

# 3) retune the sensors freely, no JSON file needed
./lab grade --task kf_fusion --controller student/kf_template.py \\\
--set gps.sigma_xy=1.2 --set imu.rate=400 --set gps.gap='[14,8]'
```

Three traps on the first start. *First:* `./lab run` spawns only the robots from `-robots` — without `-robots alice` an empty arena runs and the measurement log stays empty. *Second:* without `-truth` (or `debug_truth`) there is no `truth` topic, and the grader cannot measure. *Third:* your own node must not touch `wheel_speeds` or `cmd_vel` (section 1). Under ROS one more rule applies:

```
ros2 topic hz /alice/imu          # ~100 Hz
ros2 topic echo /alice/imu --once  # az at rest: about +9.81
ros2 topic echo /sim/config --once # sigma_xy, gap, imu-rate of the
    active profile
ros2 topic hz /alice/kf/pose       # your rate -- K4 requires at least
    10 Hz
```

4.3. Running under ROS 2 with the launch file

Without a launch file you start the simulation with `./lab sim`; with it everything comes from one line — and *every* argument is a `-set` entry for the simulator configuration (table 3). That is how you change sensor values systematically for the lab report without touching JSON files.


```

source /opt/ros/kilted/setup.bash          # if ROS is not found
                                           automatically
ros2 launch launch/kf.launch.py --show-args      # list every
                                           argument
ros2 launch launch/kf.launch.py headless:=true task:=kf_fusion \\\
    controller:=student/kf_template.py log:=messung.csv
ros2 launch launch/kf.launch.py grade:=kf_alle \\\
    controller:=student/kf_solution.py "gps_gap:=[10,14]" imu_rate:=200
ros2 launch launch/kf.launch.py rviz:=true      # view with
                                           covariance ellipse

```

Table 3: The arguments this experiment uses (selection; full list: `ros2 launch launch/kf.launch.py -show-args`).

Argument	Effect	Default
task	<code>kf_gps kf_fusion kf_kovarianz kf_dynamik</code>	<code>kf_fusion</code>
controller	your filter node, relative to the source tree	<code>student/kf_template.py</code>
grade	task or group (e.g. <code>kf_alle</code>); empty = do not grade	—
log	CSV measurement log for <code>tools/kfplot.py</code>	—
gps_rate, gps_sigma	measurements per second, spread per axis in m	profile
gps_gap	GPS outage as <code>[start, duration]</code> in s after task start	none
gps_bias_step	step bias <code>[start, duration, dx, dy]</code> in m	none
imu_rate	IMU samples per second	100 (K4: 200)
imu_accel_bias, imu_accel_walk	bias 1σ in m/s^2 , bias random walk in $\text{m/s}^2/\sqrt{s}$	$0.05 \cdot 5\text{e-}4$
imu_tilt, imu_vibration	tilt 1σ in rad (gravity leaks into a_x), vibration in m/s^2	$0.006 \cdot 0.08$
truth, seed, seconds	truth topic, noise seed, stop after s	<code>true · 1 · 0</code>

One detail with consequences: `gps_gap` and `gps_bias_step` are given in seconds *after the task starts*, not after the program starts. The grader can trigger the GPS outage at any time, and your measurement stays the same.

4.4. Three rules that save the run

First: message stamp, not wall clock. Every measurement carries `t` — the simulation time in which it was created. Use those stamps for dt , for Q and for whether a measurement has already been processed. Anyone who instead calls `time.time()` gets a wrong dt in the grading: the speed-up script `tools/fastgrade.py` packs 25 simulation seconds into one wall-clock second, and your filter then reads a 5-second gap as 200 milliseconds. *Second: you estimate, you do not drive.* In the KF tasks the grader sets `cmd_vel`, the simulation drives the command sequence from `config/tasks.json` blind. Your node

reads sensors and answers on `kf/pose` — `send_wheels()`, `publish_cmd_vel()` and `rob.truth()` are forbidden in this experiment (the grader notices this in the missing drive and the contact count), otherwise it judges a drive you never made.

Third: a new task really starts from scratch. When `rob.task()` changes, the robot is dropped at the spawn pose and the sensor profile switches. End your measurement loop then, read `rob.sensor_profile()` again and — the case that costs the most points — discard the *last* measurement of the previous task: the bus keeps the last message, and a GPS fix from before the respawn is a lie for the new task. Check `fix.t` against the start of your mission (the reference solution keeps `KF.start` for that). Two things follow from the same fact, and both are in the reference solution. The *first* measurement of the new task must also be newer than everything the bus already knew when your `mission()` began — note that baseline of stamps at the start and wait for a stamp above it, otherwise your filter initialises on the pose of the last drive, several metres away. And a step of more than about two seconds is not a prediction, it is a node that did not run: do not extrapolate five seconds of CV model over it, start the filter again on that measurement. You may repeat the gyro bias calibration — but the state and P do not belong at zero, otherwise you throw away information you only just gathered.

5. Tasks K1 to K4

All four tasks run the same logic: the grader puts your robot at the spawn pose, drives the command sequence stored in `config/tasks.json`, records `truth` and `kf/pose` and compares. Measurement only starts after the settling time (4s), so that your standstill bias calibration counts. table 4 lists the grading limits; `./lab grade -task kf_all -controller student/kf_template.py` shows you beforehand the same numbers that count in the submission.

Table 4: Grading limits of the four subtasks (source: `config/tasks.json`).

Task (points)	Sensor profile	Grading limits
K1 <i>kf_gps</i> (30)	GPS 5 Hz, $\sigma = 0.5$ m	RMSE ≤ 0.42 m, improvement ≥ 1.6 , max error ≤ 1.25 m, rate ≥ 5 Hz
K2 <i>kf_fusion</i> (30)	GPS 1 Hz, $\sigma = 0.8$ m, GPS outage 15...23s; odometry, IMU	RMSE ≤ 0.80 m, improvement ≥ 2.5 , error during the outage ≤ 1.8 m
K3 <i>kf_kovarianz</i> (20)	as K1	mean NEES in $[0.05 \dots 3.5]$, RMSE ≤ 0.42 m
K4 <i>kf_dynamik</i> (10, bonus)	GPS 5 Hz, $\sigma = 0.6$ m, IMU 200 Hz, 0.85 m/s with vibration	RMSE ≤ 0.35 m, improvement ≥ 1.8 , max error ≤ 0.9 m, rate ≥ 10 Hz

K1 — Position estimate from GPS alone. Build the CV model $\xi = (x, y, v_x, v_y)^\top$ with $F(dt)$ and the acceleration noise Q integrated over dt (eqs. (5) and (6)), then update with the GPS position *at the stamp* `fix.t`. The goal is understanding the recursion, not the bag of tricks: a single, honest KF is enough.

K2 — Fuse GPS, odometry and IMU. With 1 Hz GPS and 0.8 m spread the *absolute* error is noise-limited — so grading looks at the improvement over the raw sensor signal. The odometry `twist` helps as a prediction input (rotated into the world) and the gyro for the turn rate — both carrying a bias that first has to be measured at standstill. During the outage only the motion model is left: this is where it shows whether you ground Q in physics or guess it.

K3 — Knowing how well you estimate. The same drive as K1, but graded is the *reported* uncertainty: `kf/pose` carries σ_x, σ_y from P . Increase Q when the NEES runs above 3; decrease it when it drops below 0.3 — and write the trade-off into the lab report.

K4 — Fast drive, faithful estimate. 0.85 m/s with a sine oscillation added to the turn rate, IMU at 200 Hz. This is where it pays off that your filter runs at the *measurement rate* instead of a fixed loop rate.

Typical error patterns to recognise your implementation by: the estimate trails the measurement by a constant amount (F without dt , $Q = 0$); the covariance shrinks and never grows again (Joseph form or the P update forgotten); the robot is rotated by the gyro although the world needs the heading (sign/rotation of the body velocity); the GPS outage is bridged with an R of 10^{-6} (then the estimate follows the odometry blindly and table 4 K2 fails).

6. Evaluation and lab report

The lab report is yours — but the evaluation produces numbers you have to explain. It expects:

- **Trajectory and error signature.** One plot from `tools/kfplot.py`: truth, raw GPS, odometry, your estimate — and below it the error against truth over time. Describe *where* the error grows (curves? GPS outage? after the task switch?).
- **A Q/R table.** Which values did you use, how did you arrive at them (density $\rightarrow Q$, `gps.sigma_xy` $\rightarrow R$), and what happens when you scale them by a factor of 3 in both directions? Four measured points suffice — but they have to be measurements, not guesses.
- **NEES discussion for K3.** Your mean value, its spread within the run, and what you changed to get inside the band.
- **Measured IMU bias drift.** Average `imu` at standstill over 10 s before and after the run, report the difference per axis, and compare it with table 2. From that follows your argument for why the IMU is not integrated open loop.
- **Value of odometry during the GPS outage.** RMSE inside the outage with and without odometry prediction (two measurement logs, one concluding sentence). If you claim the model is enough on its own — measure it.
- **One sentence on the model choice.** K4 drives faster and oscillates: which state variable would you still have needed (turn rate in the state? bias as a state?), and what would it have cost?

Work from the measurement log, not from the GUI window: `-log messung.csv` writes truth, gps, odom, kf, imu and the velocities into a CSV file that you can evaluate without matplotlib too (`-list` lists the columns).

A. Formula sheet

CV model (straight line, world frame):

$$F(dt) = \begin{pmatrix} 1 & 0 & dt & 0 \\ 0 & 1 & 0 & dt \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \quad Q(dt) = q \begin{pmatrix} \frac{dt^3}{3} & 0 & \frac{dt^2}{2} & 0 \\ 0 & \frac{dt^3}{3} & 0 & \frac{dt^2}{2} \\ \frac{dt^2}{2} & 0 & dt & 0 \\ 0 & \frac{dt^2}{2} & 0 & dt \end{pmatrix}$$

with the acceleration density q in m^2/s^3 — the simulator starts with $q = 0.6$ (`kf.q_acc`), the turn rate with $q_{\text{gier}} = 0.02$.

Recursion: $\xi^- = F\xi$, $P^- = FPF^\top + Q$, $y = z - H\xi^-$, $S = HP^-H^\top + R$, $K = P^-H^\top S^{-1}$, $\xi = \xi^- + Ky$, $P = (I - KH)P^-$. For the position measurement H is the selection matrix with the two ones in the x and y columns, $R = \text{diag}(\sigma_{\text{gps}}^2, \sigma_{\text{gps}}^2)$ with σ_{gps} from `gps.sigma_xy`.

Conversions you may look up: White density d [$\text{unit}/\sqrt{\text{Hz}}$] $\rightarrow$ spread per sample $d\sqrt{\text{rate}/2}$. Bias random walk $w \rightarrow$ spread per step $w/\sqrt{\text{rate}}$, after T seconds $w\sqrt{T}$. A constant bias b over T seconds, integrated twice: position error $\frac{1}{2}bT^2$.

Consistency: $\overline{\text{NEES}} = \frac{1}{N} \sum_i [(e_{x,i}^2/\sigma_{x,i}^2 + e_{y,i}^2/\sigma_{y,i}^2)/2]$; expected value 1, a single value up to 5.99 is still unremarkable with two degrees of freedom (χ^2 , 95%). Report $\sigma_x = \sqrt{P_{11}}$ and $\sigma_y = \sqrt{P_{22}}$ — not the square root of the trace.

B. Solutions to the prelab questions

1. **IMU.** One sample: $\sigma_a = 0.002 \cdot \sqrt{100/2} = 0.014 \text{ m/s}^2$ (14 mm/s²). The factor $\sqrt{\text{rate}/2}$ comes from integrating the density over the equivalent noise bandwidth — with a sampling rate f_s that is $f_s/2$. Constant bias $b = 0.05 \text{ m/s}^2$: after $T = 10 \text{ s}$ the speed error is $\Delta v = bT = 0.5 \text{ m/s}$ and the position error $\frac{1}{2}bT^2 = 2.5 \text{ m}$. Over the same time white noise contributes only $\sigma_a T^2 N^{3/2}/\sqrt{3} \approx 0.03 \text{ m}$ ($N = 1000$ samples) — the bias dominates by two orders of magnitude. *Consequence:* the IMU serves as a *motion model* with short memory, not as a position sensor; the bias has to be estimated (or at least made small by averaging at standstill).
2. **Recursion.** With $dt = 0.5$ and $q = 0.6$:

$$FPF^\top = \begin{pmatrix} 0.04 + dt^2 \cdot 1.0 & dt \cdot 1.0 \\ dt \cdot 1.0 & 1.0 \end{pmatrix} = \begin{pmatrix} 0.29 & 0.50 \\ 0.50 & 1.00 \end{pmatrix}, \quad Q = 0.6 \begin{pmatrix} 0.0417 & 0.125 \\ 0.125 & 0.5 \end{pmatrix},$$

so $P^- = \begin{pmatrix} 0.315 & 0.575 \\ 0.575 & 1.30 \end{pmatrix}$ and $S = 0.315 + 0.25 = 0.565$. Gain $K = P^-H^\top/S = (0.315/0.565, 0.575/0.565) = (0.56, 1.02)$. Innovation $y = 2.40 - 2.00 = 0.40$,

hence $\hat{x}^+ = 2.00 + 0.56 \cdot 0.40 = 2.22$ m and $\hat{v}^+ = 0 + 1.02 \cdot 0.40 = 0.41$ m/s — the second gain entry may be larger than 1, because it acts on a velocity. Position uncertainty: $\sigma^- = \sqrt{0.315} = 0.56$ m before, $\sigma^+ = \sqrt{0.315 - 0.56^2 \cdot 0.565} = 0.37$ m after the update.

3. **Odometry drift.** The odometry pose error is the sum of all errors made so far: it is correlated in time, it grows with the distance driven, and it keeps its sign (a wheel radius that is too large stays too large). White noise with a fixed R describes an error that is forgotten with every measurement instead. Result: the filter takes each new odometry pose for an independent measurement, P shrinks toward zero, the covariance becomes too small — the NEES climbs far above 1, and during the GPS outage the estimate drifts along unchecked. The right way: use the odometry as an *input* (velocity) in the prediction, and model the remainder as process noise Q (growing with dt) or as an additional bias state.
4. **NEES.** 0.1: the reported standard deviation is on average $\sqrt{1/0.1} \approx 3.2$ times too large — the filter is needlessly cautious and trusts the measurement too little. 1: consistent, the spread of the errors matches the claim. 8: too small by $\sqrt{8} \approx 2.8$ times — overconfident, and therefore dangerous, because the application (control, safety) relies on the claim. The band $[0.05; 3.5]$ for K3 rejects 0.01 and 8 — being cautious is tolerated (a paced run of the reference solution measures 0.09 to 0.9), being over-confident is not — only 1 survives it. (The lower edge is deliberately loose — 0.15 corresponds to a claim 2.6 times too large: casual, but not dangerous. Above 3.5 it gets tight, because confident wrong claims help nobody in the lab.)

C. Topic and API overview

Access to the simulation and the topics runs through `mecanum_lab/robot_io.py`; it is the same under ROS 2 as in the stub run. These methods matter for experiment 2:

Add the topics from table 1 and, for the curious, the outside view: `ros2 topic echo /alice/imu -once`, `ros2 topic hz /alice/kf/pose`, `ros2 topic echo /sim/config -once`. If you work without ROS, you use the same topics over the in-process bus — only `ros2` does not exist there.

Table 5: Student API in experiment 2 (excerpt from `robot_io.py`).

Call	Content	Used for
<code>rob.gps()</code>	<code>Gps(t, x, y, theta)</code> — slow, scattered, <code>None</code> in the outage	K1–K4
<code>rob.odom()</code>	<code>Odom(t, x, y, theta, vx, vy, omega)</code> — 50 Hz, drifting	K2, K4
<code>rob.imu()</code>	<code>Imu(t, ax, ay, az, gx, gy, gz, roll, pitch)</code> in the body frame	K2, K4
<code>rob.scan()</code>	laser ranges (experiment 1, not graded here)	—
<code>rob.age("gps")</code>	age of the last measurement in s — <code>1e9</code> : none yet	all tasks
<code>rob.sensor_profil()</code>	active test profile as dict — the σ for R	all tasks
<code>rob.config("gps.sigma_xy", 0.5)</code>	single configuration value, with default	R , Q
<code>rob.task()</code>	name of the running task — changes with every subtask	loop end
<code>rob.running()</code>	is the simulation still running?	loop condition
<code>rob.spin(dt)</code>	process measurements without querying the wall clock	loop body
<code>rob.send_kf(x, y, theta, sx, sy, sth)</code>	your estimate with 1σ — the graded quantity	K1–K4
<code>rob.kf()</code>	what last arrived on <code>kf/pose</code> (checking your own node)	Debug
<code>rob.truth()</code>	exact pose — <i>taboo in the filter path</i> , allowed in tests	evaluation